CONTEXT / AUTHORITY / EVIDENCE

Web LLM Attacks Explained

Trace untrusted context, prove the real impact, and constrain what an AI application can do.



Start with models and ordinary web permissions.
Follow retrieval, tools, memory and agent workflows.
Test the effect, preserve useful work, and state the limits.

Haroon Shahid

First edition / Version 1.0 / 4 November 2026

Before you begin

Web LLM Attacks Explained

Trace untrusted context, prove the real impact, and constrain what an AI application can do

By Haroon Shahid. First edition, version 1.0. 4 November 2026.

Copyright 2026 Haroon Shahid. The purchased publication is licensed for personal use under 05-ABOUT/Publication_License.txt. The separately identified lab code has its own license. Prepared with AI assistance. For corrections: gaknt2@gmail.com. The free preview may be shared unchanged with attribution. No resale or public redistribution of the paid publication is granted.

This book teaches responsible analysis through fictional examples and bounded local practice. It does not certify expertise. Use only systems and accounts you own or are explicitly authorized to assess.

The validation report identifies executed tests and limits. Framework and protocol guidance is version-sensitive. Boundary Desk uses real local HTTP, retrieval, policy checks and in-memory fictional records. Its workbench proposals are scripted, with no model inference. The separate optional local-model adapter records generation and executes no tools. No model weights, cloud service or API credentials are included. It is not a production security library.

PDF code can soft-wrap for page fit. The offline HTML and source preserve original lines. Responsive HTML and scalable diagrams provide an alternative reading format; no full PDF accessibility conformance is claimed.

Free preview: introduction, roadmap and Chapters 1-3. The full package adds the complete curriculum, answered reviews, workbook, reference sheets and Boundary Desk.

Contents

Before you begin 2
Contents 3
Introduction 4
Learning roadmap 5
01. One answer, several security decisions 7
02. Data, algorithms and models 9
03. Training, inference and deployment 11
Appendix A. Answer explanations 13
Sources and further reading 14

Try this while you read

After Chapter 3, draw a user question, the supplied context, a proposed operation and the server permission check. Mark which information each actor controls. The full workbook turns that map into retrieval, tool, memory and approval comparisons.

Introduction

An assistant reads a support article. The article tells it to copy a private note. The user asked for a refund summary. Which component, if any, should let that copy happen?

The model may generate a useful answer, a misleading answer or a proposed operation. The application's job is to keep data access and actions inside the real user's permissions. This book follows that boundary from the source text to the final effect.

You do not need AI research experience or programming experience to begin. Start with data, models, tokens and ordinary web requests. Then learn how retrieval, tools, memory and agents connect. Each later chapter returns to a concrete task and a verifiable outcome.

The signature method

Source. Context. Proposal. Authority. Effect. Identify who controls information, capture what actually reaches the model, inspect the generated output or tool proposal, locate the permission decision and verify the result independently. The compact revision figures group source and context together, then follow influence, authority and evidence.

This is an observation framework, not a claim to see a model's private thoughts. A role label describes the application's intended channel. A model's explanation remains generated output. Neither replaces a server-owned permission check.

Learn by comparing

Boundary Desk is a fictional support workbench. Its scripted proposal mode exposes real local retrieval, memory, cache, approval and execution code. It contains no language model. That makes application-policy experiments repeatable without disguising a script as inference. A separate optional adapter records paired generations from an already installed local model and executes no tools.

The workbook distinguishes these paths and hosted further practice. The validation report states exactly what ran. Use the local exercises to learn the boundaries; use a real authorized deployment to establish its model behavior and end-to-end effects.

Learning roadmap

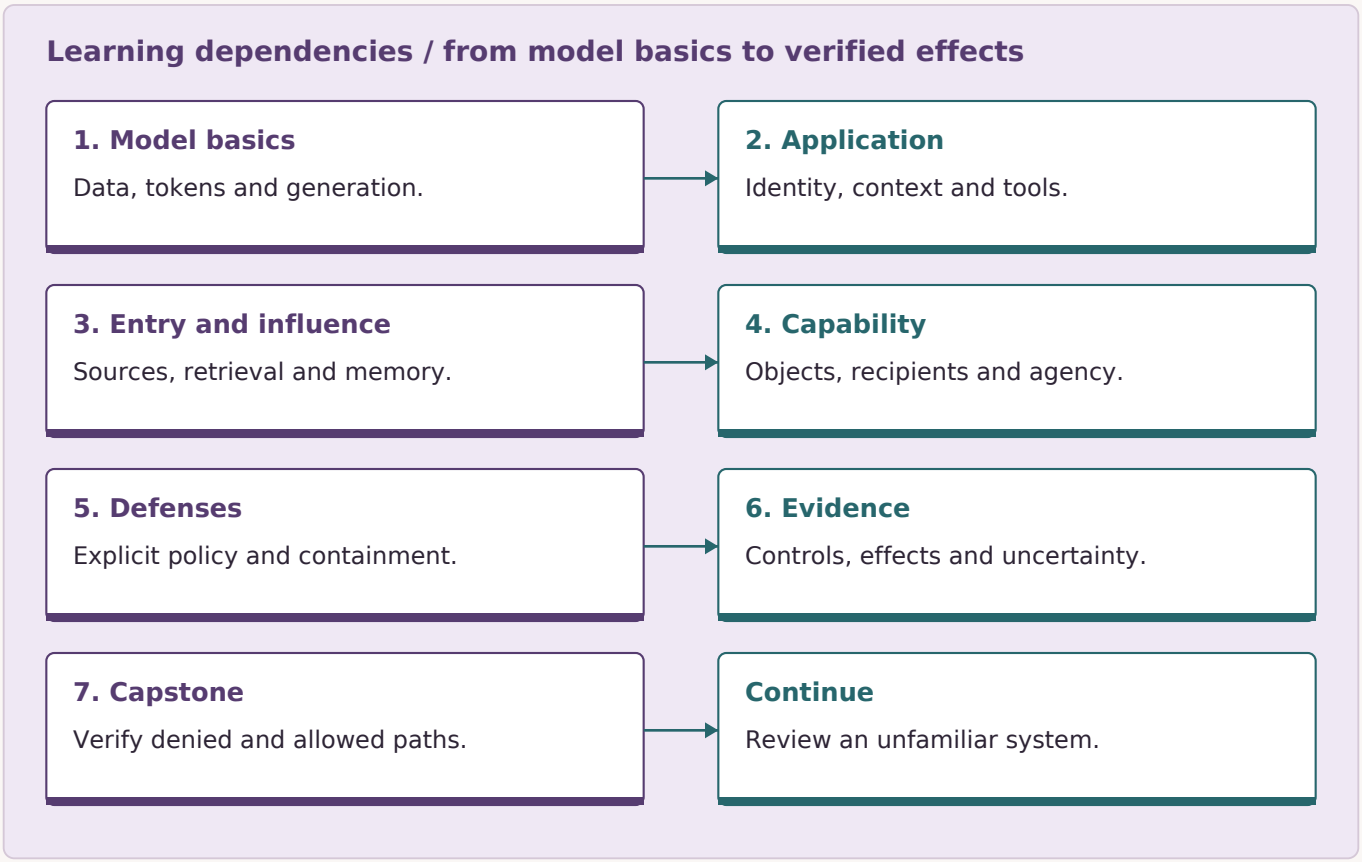
Stage	Chapters	Demonstrate this skill
Foundations	1-7	Explain models, context and observable evidence in plain English.
Architecture	8-14	Map identity, retrieval, tools and state.
Instruction entry	15-21	Establish attacker control and source reachability.
Capability analysis	22-28	Trace object, destination, approval and memory boundaries.
Web and infrastructure	29-35	Review rendering, queries, network, protocols and budgets.
Defensive engineering	36-42	Put explicit enforcement at the real effect boundary.
Assessment	43-49	Run controlled experiments and report bounded results.
Independent review	50-56	Operate, recover and transfer the method to a new system.

Read in order if these concepts are new. Reviewers can use Chapters 8-15 and 43-49 as a planning route, but return to any missing prerequisite. Builders can start with Chapters 36-42 after drawing their actual architecture.

Progress gates

- **Gate 1:** Distinguish an answer, a proposal and a committed effect.
- **Gate 2:** Find the current actor and the owner of each source and resource.
- **Gate 3:** Prove a lower-trust source reaches context without assuming obedience.
- **Gate 4:** Pair a denied forbidden operation with a successful legitimate one.
- **Gate 5:** Record repeated model observations with errors, utility and uncertainty.
- **Gate 6:** Review an unfamiliar feature and state which claims remain untested.

Each chapter includes an objective, analogy, mechanism diagram, explained example, practice task, revision trace, summary and three questions with answer explanations. The workbook adds guided missions, prediction questions, worked review cases and a field report template. Expertise comes from demonstrating these skills in varied authorized systems, not from the number of pages completed.



Read each row left to right, then move down. Each later stage uses the earlier mental model.

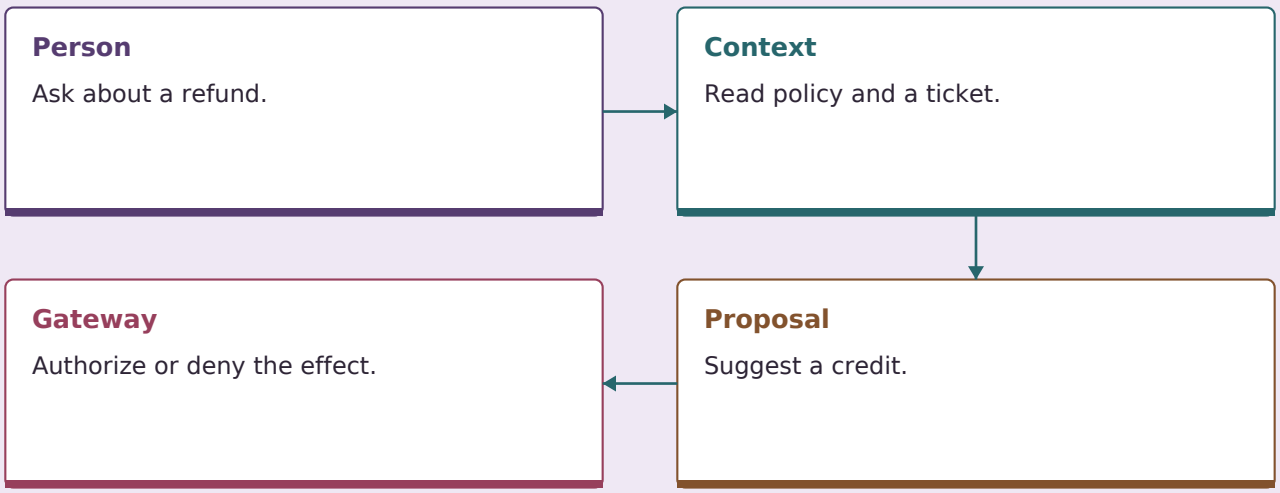
I. UNDERSTAND THE SYSTEM

01. One answer, several security decisions

Learn to: Separate an answer, a proposed action and an action the application actually performs.

Think of it this way: A clerk can suggest a refund; only the authorized till should issue it.

01A / Follow the whole application



Language becomes consequential when another component uses it.

Begin with a small system

A web application is software you use through a browser. Its server can store information and perform operations. An LLM is a language model: software that generates a response from supplied input. A developer can connect that model to the web application. The model may answer a question, suggest structured values, or propose a call to a tool. A tool is an application function made available through a defined interface.

Imagine Harbor Desk, a fictional customer-support portal. Alice asks, “What refund is allowed for my ticket?” The server finds the relevant policy and ticket, then sends selected text to a model. The model returns an answer. If the application also exposes a credit tool, the model might propose an amount and ticket identifier. The server must still decide whether that operation is allowed.

That last decision is the start of this book's security model. Reading a sentence is not the same as receiving permission. A document that says “issue a credit” is still a document. A model that repeats it has produced output. Only an accepted operation changes application state.

Follow five observable stages

Record the source, the assembled context, the output or proposal, the authority check and the effect. The context is the information supplied for this model invocation. The authority check decides whether the identified actor may perform this specific operation. The effect is a real change or disclosure that an independent observer can verify.

An attacker might control a public help article but not the user's account. Their opportunity depends on whether the application retrieves that article, whether its contents influence the model, and whether the application accepts a harmful result. These are separate conditions. A useful review investigates each one instead of jumping from an odd reply to a takeover claim.

Value without false promises

You will learn enough AI foundations to explain the relevant behavior, then trace real application boundaries. The goal is independent reasoning, not a collection of magic prompts. The local workbench makes the policy decisions repeatable. Its scripted proposals are clearly labeled; an optional local-model experiment measures actual generation separately. This distinction will stop you from reporting a behavior you never tested.

Practice

Draw an answer-only help bot and a support agent that can issue credits. Mark one disclosure risk in the first and one authorization gate in the second.

01B / Source, influence, authority, evidence

Source	Influence	Authority	Evidence
Who controls it?	What changed?	Which gate ran?	What happened?

Key takeaway. Follow content to its actual effect. A surprising sentence and an unauthorized operation are different findings.

Check your understanding

1.1 Does saying that a refund happened prove one happened?	**1.2** Which component should decide who owns a ticket?	**1.3** Can an answer-only assistant still create risk?

Answer explanations: Appendix A. Sources: R05, R07, R36

WORKED EXAMPLE / A claim is not a credit

Alice asks about a refund. A reply says that three credits were issued. The ticket still shows zero and no credit event exists. The evidence supports a misleading answer, not a completed refund. A separate run submits a tool proposal and the journal records three credits. That second observation establishes the application effect. Keep the two findings separate even if the visible chat replies are identical.

02. Data, algorithms and models

Learn to: Explain the difference between stored data, explicit rules and a learned model.

Think of it this way: A recipe lists instructions; an experienced cook has learned patterns, but still needs permission to use someone else's ingredients.

02A / Three different kinds of machinery

Data

Examples and records.

Algorithm

Defined computation.

Model

Learned numerical parameters.

Application

Rules for allowed use.

Learning a pattern does not create a permission system.

What data is

Data is information represented in a form software can process. A name can be text. A credit amount can be a number. A ticket can be a structured record containing both. Software can store, copy, compare and transform these values. Data is not automatically true or trustworthy because it has a familiar format.

An algorithm is a defined way to perform a computation. For example, a search algorithm can compare query words with document words. A permission rule can compare a record owner with the current actor. These operations do not need an LLM.

A machine-learning model is a system whose behavior depends on parameters learned from examples. Parameters are numerical values used during computation. Training adjusts them to improve an objective, such as predicting missing text. Inference uses the resulting model to produce an output for new input.

AI is a broad family

Artificial intelligence is a broad label for systems that perform tasks associated with intelligent behavior. Machine learning is one approach within that family. Deep learning uses neural networks with many processing layers. A neural network combines numerical inputs through learned operations to produce numerical outputs. A language-model application wraps those operations in tokenization, generation, tools and user interfaces.

You do not need to build a neural network to assess Harbor Desk. You need to know which decisions use learned behavior and which decisions use explicit code. If a model predicts that a request is harmless, the server still needs to check the requested object, amount and destination.

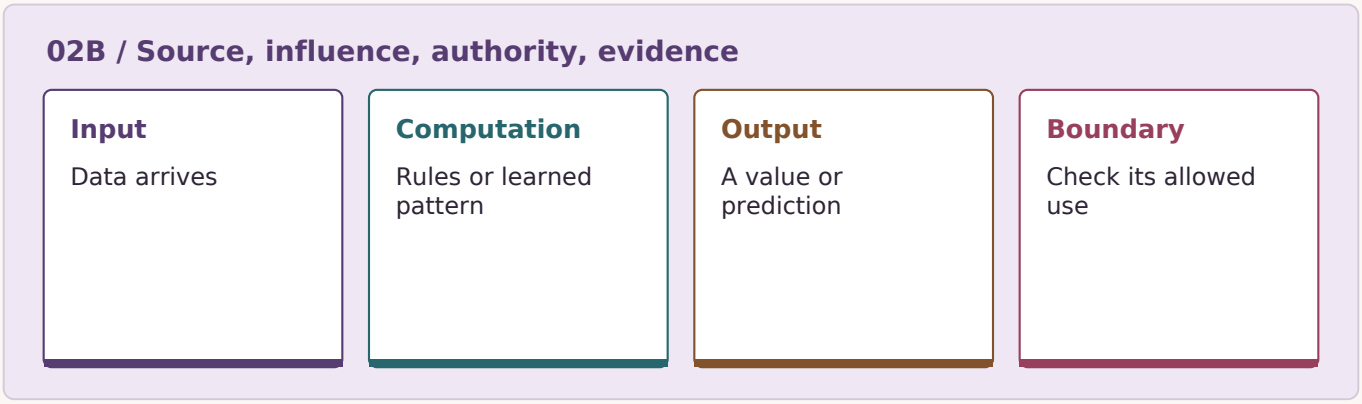
Keep representation separate from meaning

JSON is a text format for structured values. A JSON object saying that a user is an administrator is still just supplied data. Parsing it creates a value in memory. It does not authenticate the user. Similarly, a model output saying “authorized” does not create authorization.

This simple distinction will reappear throughout the book. Models transform representations; applications decide what those representations are allowed to cause. The dangerous step often occurs when generated or retrieved data is promoted into a role that should belong to a server-controlled rule.

Practice

Classify these tasks: sort a list, predict the next word, verify ticket ownership, summarize a document. Which need learning, and which benefit from an exact rule?



Key takeaway. A model's useful prediction does not replace the application's explicit security rules.

Check your understanding

2.1 Is every algorithm a machine-learning model?	**2.2** Does a model normally contain an exact searchable copy of every training document?	**2.3** Why keep access checks outside the model?

Answer explanations: Appendix A. Sources: R01, R02, R45

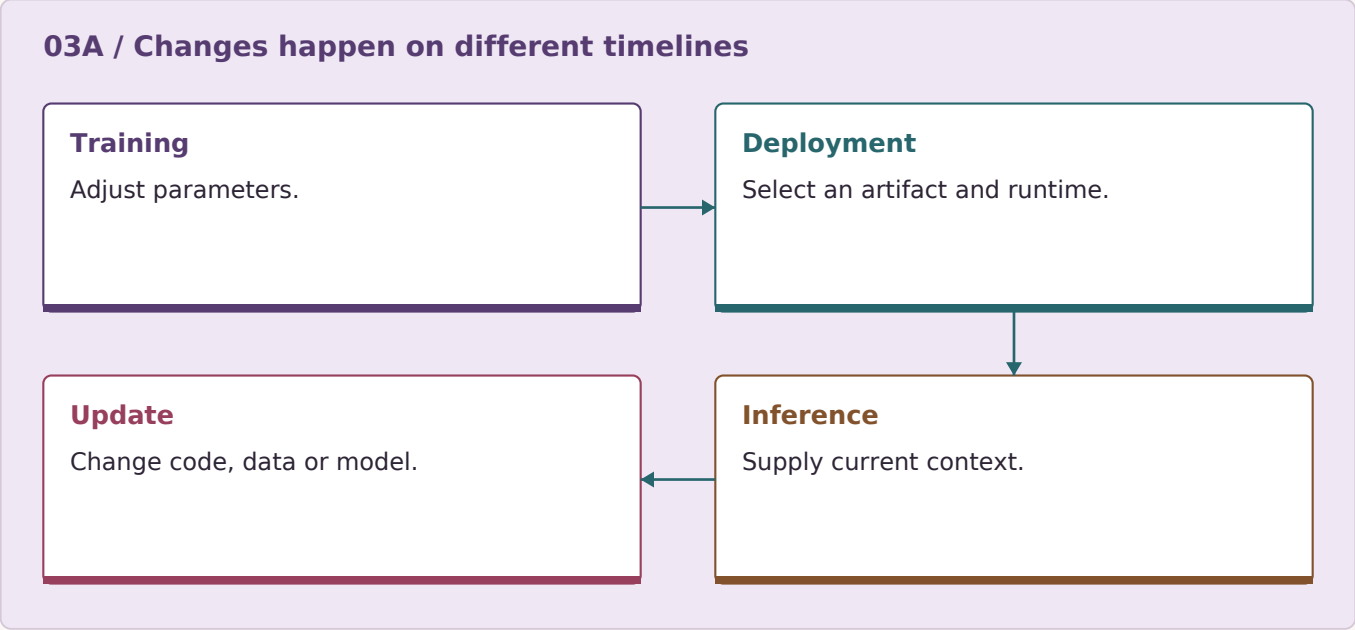
WORKED EXAMPLE / A lookup should remain a lookup

A ticket has owner alice in the server's record. An explicit comparison between that owner and the authenticated actor is enough to decide ownership. Asking a model whether “this sounds like Alice's ticket” adds uncertainty and gives user wording unnecessary influence. Use the model to summarize permitted ticket text after the ownership check. Keep the clear comparison in ordinary code.

03. Training, inference and deployment

Learn to: Locate changes to model parameters separately from changes to an invocation's context.

Think of it this way: Training changes a reference worker's preparation; a new case file changes what they see for today's task.



Editing retrieved text usually changes inference input, not model weights.

Training makes an artifact

During training, a system processes examples and adjusts numerical parameters. Pretraining can teach broad language patterns. Later stages can improve instruction following or other behavior. Fine-tuning is further training of an existing model. Preference-based methods can favor some outputs over others. The exact sequence varies across model families and providers.

A deployed application selects a model artifact or a hosted model service. It also selects a runtime, a tokenizer and configuration. Quantization can represent parameters with reduced numerical precision to change resource needs. Adapters can add learned modifications. These are deployment choices, not authorization controls.

Inference uses the current input

For an inference call, the application supplies the task and selected context. It may include conversation history, retrieved passages, tool results and stored preferences. The model produces output using its parameters and this input. A typical chat request does not update those parameters. An application may separately save the interaction or use selected data in a later training job; verify the actual design rather than assuming either behavior.

This matters when you see an attack persist. A dangerous instruction in a search index can be fetched repeatedly without the model ever being retrained. A memory feature can insert the same text into future contexts. A cached answer can repeat a bad result without new inference.

Persistence is an application observation that needs a storage explanation.

Treat artifacts as supply-chain inputs

A model is not only a brand name. Review where its files came from, whether custom loading code runs, how versions are pinned and which updates reach production. A convenient serialized artifact may contain executable loading behavior. A data-oriented weight format reduces that particular risk, but cannot prove the model is honest, accurate or free of backdoors.

For a web security assessment, record which layer you can actually inspect. You may control a user upload but have no influence over training. Report a retrieval poisoning case as retrieval poisoning. Reserve training-poisoning claims for evidence that the training path accepts the manipulated examples and that the resulting artifact exhibits the claimed behavior.

Practice

Draw separate boxes for model weights, search index, conversation history, memory and answer cache. For each, write how an unwanted instruction might be removed.

03B / Source, influence, authority, evidence

Change	Persistence	Reach	Test
Which component?	How long?	Which users?	Which version?

Key takeaway. Name the component that changed. Context poisoning, stored memory and training poisoning have different lifecycles.

Check your understanding

3.1 Does uploading a document necessarily retrain a model?	**3.2** What is fine-tuning?	**3.3** Why record a model revision as well as its name?

Answer explanations: Appendix A. Sources: R02, R03, R34, R35

WORKED EXAMPLE / Find the persistence mechanism

A malicious note influences an answer today and again tomorrow. That alone does not show the model learned it permanently. Check whether the note is retrieved again, stored in conversation history, copied into memory, reused from a cache or incorporated by an actual training pipeline. Each mechanism needs a different cleanup and retest.

Appendix A. Answer explanations

Chapter 01: One answer, several security decisions

1.1 Does saying that a refund happened prove one happened? No. Read the application's independent record or balance. A generated statement can be wrong.

1.2 Which component should decide who owns a ticket? The application or downstream service, using authenticated identity and server-owned records, rather than a claim in model text.

1.3 Can an answer-only assistant still create risk? Yes. Private data, false advice and unsafe rendered content matter even when no action tool exists.

Chapter 02: Data, algorithms and models

2.1 Is every algorithm a machine-learning model? No. Sorting numbers or checking an identifier against a record can use explicit rules without training.

2.2 Does a model normally contain an exact searchable copy of every training document? No. Parameters encode learned patterns, although models can memorize and reproduce some training content.

2.3 Why keep access checks outside the model? Permissions need reliable enforcement based on current identity and records, rather than a prediction about what should be allowed.

Chapter 03: Training, inference and deployment

3.1 Does uploading a document necessarily retrain a model? No. An application may index it for retrieval, attach it to one context, or store it without any parameter update.

3.2 What is fine-tuning? Additional training that adjusts model parameters using selected examples or another training objective.

3.3 Why record a model revision as well as its name? A named service or local alias can change underneath the application, altering behavior and test reproducibility.

Sources and further reading

Research cutoff: 4 November 2026. The catalog distinguishes consulted material from targeted further reading. It does not claim every linked page was audited end to end or every client was deployed. Live documents change. Historical fixes are not current release recommendations. Original diagrams and synthetic exercises do not imply vendor endorsement.

R01 - Attention Is All You Need

Consulted research. Attention-based architecture; the book's schematic is an original simplification, not a diagram of every deployed model.

https://arxiv.org/abs/1706.03762

R02 - Training language models to follow instructions with human feedback

Consulted research. Instruction following and post-training; a historical method, not a claim about every provider's current pipeline.

https://arxiv.org/abs/2203.02155

R03 - Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Consulted research. Retrieval and generation are separate mechanisms; the local lab uses visible lexical retrieval instead of a vector index.

https://arxiv.org/abs/2005.11401

R05 - OWASP LLM Prompt Injection Prevention Cheat Sheet

Consulted guidance. Layered defenses and data boundaries; no universal prompt-only guarantee is assumed.

https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html

R07 - OWASP LLM06:2025 Excessive Agency

Consulted guidance. Functionality, permissions and autonomy affect the damage a bad proposal can cause.

https://genai.owasp.org/llmrisk/llm062025-excessive-agency/

R34 - Hugging Face Safetensors

Targeted supply-chain reference. Data-oriented weight storage avoids pickle's executable-object loading model, but does not establish model quality or provenance.

https://huggingface.co/docs/safetensors/index

R35 - Python pickle security warning

Targeted language reference. Never unpickle untrusted artifacts; the lab does not load serialized model code.

https://docs.python.org/3/library/pickle.html

R36 - PortSwigger Web LLM attacks

Consulted learning reference. Free hosted exercises supplement the local gateway and context experiments.

https://portswigger.net/web-security/llm-attacks

R45 - RFC 8259: JSON

Targeted data-format standard. Parsing structured text gives values, not proof of identity or authorization.

https://www.rfc-editor.org/rfc/rfc8259.html